# Resumen ICS Parcial 1

## Software

El software no es solo código, es importante incorporar que el software es un set de programas y la documentación que acompaña.

## Estado Actual y Antecedentes. La Crisis del Software.

El concepto “ingeniería de software” se propuso en una conferencia realizada para discutir lo que
entonces se llamaba la “crisis del software”. En la década de 1960, se observó una brecha entre la creciente demanda de software y la capacidad de producirlo de manera eficiente y fiable. Se volvió claro que los enfoques individuales al desarrollo de programas no escalaban hacia los grandes y complejos sistemas de software. Estos no eran confiables, costaban más de lo esperado y se distribuían con demora.

A lo largo de las décadas de 1970 y 1980 se desarrolló una variedad de nuevas técnicas y métodos de ingeniería de software, tales como la programación estructurada y el desarrollo orientado a objetos. Se perfeccionaron herramientas y notaciones estándar y ahora se usan de manera extensa.

La ingeniería de software es crucial en la sociedad moderna. Existen una variedad de sistemas de
software, cada uno con necesidades y enfoques de ingeniería específicos. La complejidad y
demandas cambiantes en el desarrollo de software a menudo conducen a desafíos y "fallas de
software", que se pueden atribuir al rápido avance de las tecnologías y a la subestimación de la
importancia de la ingeniería de software

## Tipos De Software

**SYSTEM SOFTWARE:** este tipo de software se encarga de controlar y gestionar los recursos del hardware de una computadora, así como de

proporcionar una interfaz para que los usuarios interactúen

**UTILITARIOS:** Se refiere a un subconjunto de software de sistema que proporciona utilidades o herramientas adicionales para mejorar la funcionalidad del sistema operativo y el rendimiento de la computadora

**SOFTWARE DE APLICACIÓN:** Este software se utiliza para realizar tareas específicas en una computadora, como procesar texto, crear presentaciones o navegar por internet. Se enfoca en satisfacer las necesidades de los usuarios

# SOFTWARE ≠ MANUFRACTURA

5 Razones para no comprar Software con Manufactura

- El software es menos predecible, no como los productos tangibles en una línea de producción
- Casi ningún producto de software es igual a otro, ya que de alguna manera los requerimientos o necesidades de los usuarios que van a hacer uso de ese software van a ser distintas.
- No todas las fallas en software son errores. El tratamiento de errores en software es totalmente distinto en software que en la producción.
- El software no se gasta. Cuando hablamos de productos, normalmente, hablamos de tiempo, que al pasar del tiempo el producto se desgasta, esto no sucede con el software
- El software (obviamente) no está gobernado por las leyes de la física.

# PROBLEMAS AL DESARROLLAR/CONSTRUIR SOFTWARE

- MÁS TIEMPOS Y COSTOS QUE LOS PRESUPUESTADOS
- QUE LA VERSIÓN FINAL DEL PRODUCTO NO SATISFAGA LAS NECESIDADES DEL CLIENTE
- PROBLEMA EN LA ESCALABILIDAD Y/O ADAPTABILIDAD DEL SOFTWARE
- MALA DOCUMENTACIÓN
- MALA CALIDAD DEL SOFTWARE

# Software Exitoso

- Involucramos al usuario
- Existe un apoyo de la gerencia
- Requerimientos correctamente enunciados o entendibles
- Personal competentemente capacitado
- Planeamiento adecuado
- Expectativas realistas o alcanzables
- Hitos intermedios

# Software Fracaso

- Requerimientos incompleto o cambiantes
- Falta de involucracion del usuario
- Recursos insuficientes
- Expectativas pocos realistas
- No existe mucho apoyo de la gerencia

# Ingeniería del Software

La **ingeniería** de **software** es el establecimiento y uso de principios fundamentales de la ingeniería con el objetivo de desarrollar, ejecutar y mantener en forma económica software que sea confiable y que trabaje con eficiencia en máquinas reales.
Este enfoque abarca todo el ciclo de vida del software, desde su **concepción** y **diseño** hasta su
**implementación**, **pruebas** y **mantenimiento** en entornos reales. Uno de los pilares fundamentales de la ingeniería de software es el compromiso con la calidad, que implica no solo desarrollar software funcional, sino también software confiable y eficaz que satisfaga las necesidades y expectativas de los usuarios.

# Disciplinas que conforman la Ingeniería de Software

La Ingeniería de Software abarca diversas disciplinas que trabajan de manera conjunta para desarrollar, gestionar y mantener sistemas de software eficientes y de alta calidad

- **Disciplinas técnicas**: Se centran en el conocimiento y la aplicación de principios y técnicas específicas para desarrollar software de manera efectiva. Incluye definir requerimientos, el análisis, diseño, implementación y pruebas.
- **Disciplinas de gestión:** Relacionadas con la planificación, coordinación y control de los
  recursos y actividades del proyecto de desarrollo de software. Implica estimar recursos, programar actividades y establecer objetivos para guiar la ejecución del proyecto, además de monitorear el avance del proyecto.
- **Disciplinas de soporte:** engloban aquellas áreas que proporcionan apoyo para mejorar la
  calidad, seguridad y usabilidad del software desarrollado. Dentro de estas podemos
  encontrar la gestión de configuración, el uso de métricas, etc.

# Proceso Definido

Se basan en una concepción basada en el modelo industrial, en la cual tengo un conjunto de inputs que van a ser utilizados en un conjunto dado de actividades, que van a producir el mismo resultado o salida siempre que los inputs sean los mismos

No podemos simplificar el software a una entrada de inputs que siendo utilizado para determinadas aplicaciones produce el mismo output

Asume que podemos repetir el proceso indefinidamente y obtener los mismos resultados.

# Proceso Empirico

Viene del empirismo, es aquella corriente en la cual centramos el foco en la experiencia. No solo la experiencia del usuario sino también la experiencia en

el negocio, la experiencia técnica, en cómo voy a manipular las herramientas y aprender de los errores para poder aplicarlo

Nos basamos en captar la experiencia de los usuarios involucrados en la producción para maximizar la calidad de lo que estamos produciendo

Las variables no pueden ser definidas o controladas en su totalidad, ya que, estamos dentro de un ambiente complejo donde quizás no todo lo que está en juego está bajo nuestro control y esto nos implica la necesidad de adaptación para poder garantizar la producción de un software de calidad

Se basan en la experiencia y el aprendizaje, y la principal diferencia con los procesos definidos, es la forma en la cual se puede mejorar el proceso. Mientras que en el proceso definido el punto de mejora es el central, el proceso empírico no tiene un punto de mejora tan tangible, voy a tener que ir trabajando sobre distintos aspectos

# Ciclos de Vida

Son la serie de pasos a través de los cuales un producto o un proceso progresa.

El ciclo de vida es una abstracción que me va a definir cuáles son las etapas (fases) y el orden de cada una de ellas.

No me dice que tengo que hacer, ni quien lo va a hacer, ni las herramientas ni los procedimientos, sino, solo las fases y en qué orden se van a ejecutar aportando informacion de cada una.

# CICLOS DE VIDA DE PRODUCTO Y PROYECTO DE SOFTWARE

El ciclo de vida del **producto** empieza con la primera versión del mismo y termina cuando
deja de utilizarse por perder su valor para el cliente.

El **proyecto** tiene un objetivo en concreto, por ejemplo, lanzar una versión de un producto o
agregar algo a un producto ya existente. Cuando se cumple ese objetivo, finaliza el ciclo de
vida del proyecto

# Tipos de Ciclos de Vida

**SECUENCIAL (Ej. Cascada o Tradicional):** Se basan en ejecutar una etapa después de otra sin retorno generalmente. Sugiere un enfoque **sistemático** y **lineal** para el desarrollo del software, que comienza con la especificación de los requerimientos y avanza a través de planeación, modelado, construcción y despliegue, para concluir con el apoyo del software terminado. Es aplicable cuando los requerimientos para cierto problema se comprenden bien

🔹 **Ventajas**

- Sencillo de entender y aplicar.
- Buena documentación y control en cada fase.
- Útil cuando los requerimientos están bien definidos y estables.

🔸 **Desventajas**

- Poco flexible frente a cambios.
- Errores detectados tarde hacen que sea mas costoso.
- El cliente recién ve resultados al final → poca retroalimentación temprana.

**ITERATIVO/INCREMENTAL (Procesos Empíricos / Ágiles):** Ejecuta una serie de avances que en forma progresiva dan más funcionalidad al cliente conforme se le entrega cada incremento. Las actividades de especificación, desarrollo y validación están entrelazadas en vez de separadas, con rápida retroalimentación entre ellas.

Esto permite que en cada una de las iteraciones capitalicemos lo aprendido, mejoremos la experiencia del equipo y realicemos otra iteración para incrementar el producto

Se recomienda cuando los requerimientos iniciales del software están razonablemente bien
definidos, pero el alcance general del esfuerzo de desarrollo imposibilita un proceso linea o desarrollar rapidamente una funcionalidad y mejorarla en posteriores incrementos.

🔹 **Ventajas**

- Permite entregas parciales de valor al cliente.
- Fomenta retroalimentación y aprendizaje en cada iteración.
- Reduce riesgos al detectar errores antes.

- Mejora continua y adaptación a cambios.

🔸 **Desventajas**

- Requiere mayor involucramiento del cliente.
- Puede ser más costoso si no se gestiona bien.
- Riesgo de que las iteraciones nunca terminen (si no hay buena planificación).

**RECURSIVO (Ej. Modelo en Espiral):** Reconocen la naturaleza cambiante de los requisitos y las necesidades del cliente a
lo largo del tiempo y se caracterizan por la manera en la que permiten desarrollar versiones
cada vez más completas del software. Se utiliza para casos particulares como proyectos de alto riesgo. Se basa en tomar una funcionalidad específica y centrarse en el desarrollo en esa funcionalidad, para continuar con otra

🔹 **Ventajas**

- Excelente para proyectos de alto riesgo.
- Incorpora explícitamente análisis de riesgos en cada vuelta.
- Permite refinar progresivamente el producto.

🔸 **Desventajas**

- Más complejo de gestionar.
- Costoso y requiere experiencia del equipo.
- Puede ser excesivo para proyectos pequeños o simples.

# Ciclos de vida y su influencia en la Administración de Proyectos de Software.

Un **proceso de software** es una serie de actividades relacionadas que conducen a la elaboración de un producto de software. Los procesos de software son complejos y se apoyan en personas con capacidad de juzgar y tomar decisiones.

No hay un proceso ideal, las organizaciones diseñan su propio proceso de desarrollo de software.

Los procesos han evolucionado para beneficiarse de las capacidades de la gente en una
organización y de las características específicas de los sistemas que se están desarrollando.

Si se quiere administrar, un proceso de software debe ser explícitamente modelado. Un **modelo** de **proceso** o un **ciclo** de **vida** de software es una representación simplificada de este proceso

# Procesos de Desarrollo de Software

Un **proceso de software** se define como un conjunto estructurado de actividades que, a raíz de un conjuntode entradas, producen una salida. Estas actividades varían dependiendo de la organización y el tipo de sistemaque debe desarrollarse

Hay diferentes percepciones de lo que es un **proceso de desarrollo de software**, pero el objetivo siempre va a ser obtener un producto o servicio de software a partir de distintos inputs. No solo vamos a tener como entrada o input los requerimientos sino que también vamos a tener a las personas y a su vez materiales, energía, equipamiento

# Relacion entre Proceso, Proyecto y ciclo de vida

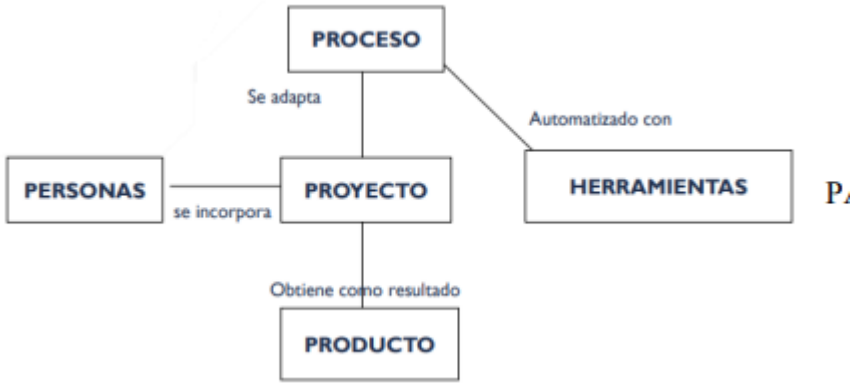
El **Proceso** define las actividades, el **Ciclo de Vida** les da un orden temporal, el **Proyecto** es el entorno real donde se ejecutan esas actividades

# Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto

Cada proyecto tiene necesidades diferentes, no existe un modelo de ciclo de vida que traiga mejores resultados para cualquier proyecto. Para seleccionar el modelo de ciclo de vida más conveniente para determinado proyecto, nos podemos hacer diversas preguntas

- ¿Es probable que los requerimientos identificados cambien a medida que avance el
  proyecto? El prototipado evolutivo trabaja con poca identificación de los requerimientos y permite modificaciones a medio camino, no así el modelo en cascad
- ¿Comprendemos bien la arquitectura del sistema? Si la arquitectura es modular y fácilmente descomponible, un enfoque iterativo o incremental puede ser más adecuado. Si la arquitectura es compleja y monolítica, un enfoque en cascada podría ser más apropiado
- ¿Cuántos riesgos conlleva el proyecto? El modelo en espiral es conveniente para la gestión de riesgos, ya que incluye una fase dedicada específicamente al análisis de riesgos

# Caracteristicas/Componentes de un Proyecto de Software/Sistemas de informacion




El **proyecto** es un esfuerzo temporal y unidad de gestión, y necesita de personas, recursos y procesos para poder existir. A través del mismo, administro los recursos que necesito y las personas que van a formar parte del mismo, para obtener como resultado un producto de software

El **proceso** es una definición abstracta que se materializa a través de los **proyectos**, en donde se adapta a las necesidades concretas del mismo. En un

proceso se expresa en forma teórica lo que se debe hacer para hacer software y debo adaptarlo al proyecto

Un proyecto de software tiene que cumplir con ciertas características:

- **Duracion Limitada**: Son temporales, cuando se alcanza el/los objetivos el proyecto termina
- **Orientado a Objetivos:** Los proyectos stán dirigidos a obtener resultados y esto se refleja a través de objetivos que guían el mismo, estos deben ser claros, unicos y alcanzables
- **Conjunto de tareas interrelacionadas que requieren la asignación de recursos y esfuerzo:** Las tareas dependen una de otra, tienen precedencia y a su vez tienen asociados/designados recursos y esfuerzos, lo cual hace que la gestión de proyectos sea una tarea compleja
- **Son Unicos**: Por más similares que sean, los proyectos tienen características que los hacen
  único

**Administración de proyectos**

Es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto. Implica tener el trabajo hecho en tiempo, con el presupuesto acordado, habiendo satisfecho los requerimientos. Incluye:

- Identificar requerimientos
- Establecer objetivos claros y alcanzables
- Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los
  involucrados (stakeholders).

# La Triple Restriccion

La triple restricción es una de las bases de la gestión tradicional de proyectos. Plantea que en un proyecto vamos a tener una restricción de tiempo (como un cronograma con una fecha específica), una restricción de presupuesto y un alcance. Estas tres variables están íntimamente relacionadas de manera tal

que, si modifico una de ellas, voy a tenerque hacer variar alguna de las otras o ambas

- Alcances:  Son los requerimientos que el cliente expresa, el objetivo, lo que quiero alcanzar.
- Tiempo: Tiempo que debería llevar completar el proyecto
- Costos: Recursos necesarios para llevar adelante el proyecto

# Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.

## 1. Proceso

- Es la **plantilla o marco general** que define cómo se desarrolla software en una organización.
- Incluye actividades, métodos, prácticas y herramientas que guían el desarrollo.
- Es **abstracto**: no construye directamente nada, sino que marca el **camino estándar**.

---

## 2. Proyecto

- Es la **instanciación concreta del proceso** para alcanzar un objetivo específico.
- Tiene **inicio y fin definidos**.
- Requiere asignación de recursos, roles y tareas.
- Gestiona la **triple restricción** (alcance, tiempo, costo).
- Resultado: un **producto parcial o versión** del software.

---

## 3. Producto

- Es el **resultado final** del proyecto de desarrollo.
- Incluye el software y su documentación asociada.

- Tiene un **ciclo de vida más largo que el proyecto**, ya que continúa en mantenimiento y evolución hasta que deja de tener valor.
- Refleja la calidad tanto del proceso seguido como de la gestión del proyecto.

# Unidad 2 - Manifesto Agil

# Requerimientos Agiles

Ágil es una ideología con un conjunto definido de principios que guían el desarrollo del producto. Los métodos ágiles son adaptables en lugar de predictivos y orientados al cliente en lugar de al proceso

Se plantea que los requerimientos se irán descubriendo a medida que avanza el proyecto y se busca definir "sólo lo suficiente" para comenzar a trabajar con lo mínimo necesario y obtener retroalimentación lo antes posible para mejorar continuamente.  En lugar de especificar todos los requerimientos desde el inicio, el desarrollo de requerimientos está enfocado en trabajar en conjunto con el cliente.

Los requerimientos se expresan en forma de historias de usuario, que son breves descripciones de los requisitos del cliente que se escriben en lenguaje natural y se enfocan en las necesidades del usuario. Estas historias se utilizan para definir el alcance de cada iteración del proyecto y se priorizan en función del valor que aportan al producto

Una de las razones por las cuales necesitamos determinar solo lo que suficiente, es porque todos los productos de software tienen un desperdicio significativo, esto quiere decir que, de todas las funcionalidades desarrolladas, es muy alto el porcentaje de aquellas que el cliente/usuario no usa

La gestión ágil de requerimientos intenta contrarrestar esta situación, priorizando solo aquellas funcionalidades que aportan valor al cliente.

Para lograr esto, la gestión ágil utiliza el concepto de dueño del producto (Product Owner) como una figura clave en el proceso de desarrollo. El PO es el

responsable de identificar las necesidades y prioridades del negocio y, en consecuencia, de priorizar los requerimientos del producto.

Estos requerimientos se organizan en una lista priorizada y ordenada, llamada Product Backlog.  Los requerimientos más importantes se encuentran en la parte superior de la lista, mientras que los menos importantes se encuentran en la parte inferior

LA GESTIÓN ÁGIL DE LOS REQUERIMIENTOS APUNTA A TRABAJAR Y A PONER EL ESFUERZO EN LO QUE NOS DA VALOR, Y A TRABAJARLO EN EL MOMENTO ADECUADO

A MEDIDA QUE AVANZAMOS, HAY REQUERIMIENTOS QUE PUEDEN SER REMOVIDOS, OTROS QUE PUEDEN SUBIR Y TOMAR UN MAYOR VALOR EN LA PILA Y OTROS QUE PUEDEN BAJAR, O BIEN PUEDEN AGREGARSE REQUERIMIENTOS A LA PILA

# Tipos de Requerimientos

- DE NEGOCIO: Son los de más alto nivel, nos referimos a cuales son los requerimientos en
  términos de la visión del negocio
- DE USUARIO: Tienen que ver concretamente con los requerimientos de usuario final
- FUNCIONALES: Podríamos relacionarlo 1 a 1 con los casos de uso en la gestión tradicional
- NO FUNCIONALES: Suelen ser los más ocultos, que el cliente no tiene en claro y debo
  especificarlos. Un requerimiento no funcional puede hacer que el software que estoy desarrollando no sirva (IMPORTANCIA FUNDAMENTAL)
- DE IMPLEMENTACIÓN: Se suele decir que están dentro de los no funcionales pero tienen que ver con cuestiones de restricción o implementación específicos.

A raíz de esta clasificación, queremos identificar aquellos requerimientos que tienen que ver con el dominio del problema y luego de la solución y por ende con los requerimientos específicos de software.

Esta distinción es muy importante ya que es fundamental entender las necesidades del usuario y las restricciones del contexto en el que se utiliza el

software. Esta comprensión permite a los equipos de desarrollo adaptarse rápidamente a los cambios y priorizar las características más importantes del software

# Conceptos relacionados a los Principios Agiles

- LOS CAMBIOS VAN A EXISTIR TODO EL TIEMPO DE FORMA CONSTANTE: Sabemos que hay cambio, por lo que se busca gestionar los requerimientos orientados a poder aceptar el cambio. Si no hay cambios significa que el producto no se esta usando
- DEBEMOS TENER UNA MIRADA DE TODOS LOS QUE ESTÁN INVOLUCRADOS: Todos los que tienen algo para decir del producto
- EL USUARIO DICE LO QUE QUIERE CUANDO RECIBE LO QUE PIDIÓ: Es por esto que las iteraciones cortas y las entregas continuas son esenciales, ya que permiten una retroalimentación y un enriquecimiento para futuras entregas
- NO HAY TÉCNICAS NI HERRAMIENTAS QUE SIRVAN PARA TODOS LOS CASOS: A veces se necesita trabajar con más prototipos o modelos, otras veces con técnicas sencillas es suficiente, pero hay que ver en cada caso que es lo que conviene.
- LO IMPORTANTE NO ES ENTREGAR UNA SALIDA, UN REQUERIMIENTO, SINO ES ENTREGAR UN RESULTADO, UNA SOLUCIÓN DE VALOR: Vamos a apostar a entregar una solución que le de valor al cliente, con esto nos referimos a algo que sea útil para el negocio

# Principios Agiles Relacionados a los Req Agiles

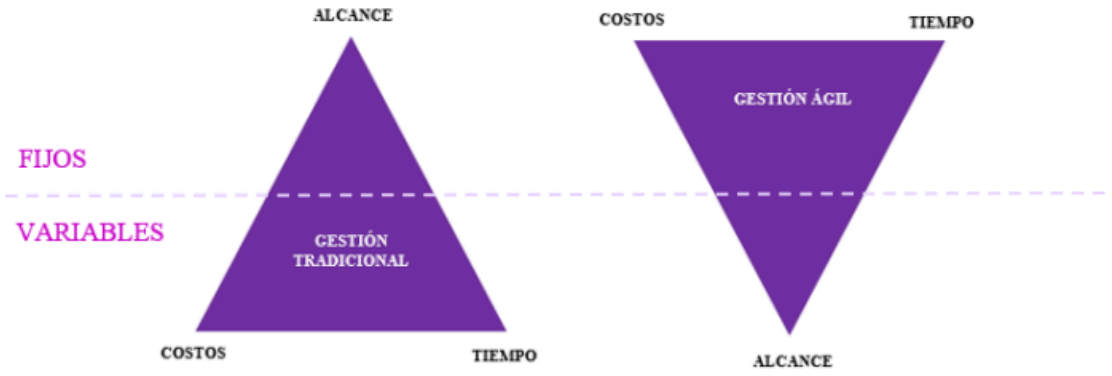
1. La prioridad es satisfacer al cliente a través de releases tempranos y frecuentes (2 semanas a un mes)
2. Recibir cambios de requerimientos, aun en etapas finales.
3. Técnicos y no técnicos trabajando juntos todo el proyecto: El PO es parte del equipo, porque si los no técnicos no son parte del equipo del proyecto, difícilmente pueda identificar qué da valor al negocio

4. El medio de comunicación por excelencia es cara a cara.
5. Las mejores arquitecturas, diseños y requerimientos emergen de equipos auto organizados

# Triangulo de Hiero VS Triangulo Agil

El triángulo de hierro es un modelo utilizado en la gestión tradicional de proyectos que establece que los tres elementos fundamentales en cualquier proyecto son el alcance, el tiempo y el costo. Según este modelo, cualquier cambio en uno de estos elementos puede afectar los otros dos. En la gestión tradicional de proyectos, se busca establecer un plan detallado y fijo en cuanto a estos tres elementos antes de comenzar el proyecto, y se trabaja para asegurar que se cumplan en el transcurso del proyecto.

Por otro lado, en las metodologías ágiles, se considera que el alcance, el tiempo y el costo son variables flexibles que pueden adaptarse y cambiarse a lo largo del proyecto. En lugar de establecer un plan fijo al inicio del proyecto, las metodologías ágiles utilizan iteraciones cortas y regulares para adaptar el proyecto según las necesidades y los requisitos cambiantes.
La triple restricción tradicional es considerada en menor medida ya que la gestión ágil establece que los tres elementos fundamentales son el valor, la calidad y la restricción del tiempo. En este modelo, se busca maximizar el valor que se entrega al cliente, asegurar la calidad del producto y cumplir con los plazos establecidos, en lugar de centrarse en alcance y costo




# User Stories

Es una técnica para capturar requerimientos que se basa en funcionalidad relativamente pequeña que será valiosa para los usuarios del software. Se llama “User Stories” porque es una herramienta o técnica que la manera de utilizarlo es como si contáramos una historia y expresan la intencion de querer hacer algo.

Las US son Multiproposito porque modelan distintas cosas

- Una necesidad del usuario. Especifica que necesita el usuario
- Una descripción del producto. Describe las características específicas del usuario.
- Un ítem de planificación: Al priorizar las historias, cada una de ellas es un ítem de planificación

Las US que están en el tope de pila del Producto Backlog, son aquellos que tiene que ser modelados con mayor nivel detalle y a los que debemos prestar más atención.

Plantean que contar una historia es una buena manera de precisar qué es lo que le da valor a nuestro involucrado y qué es lo que espera del software que vamos a construir.

# 3 “C” De Una US

- **Conversacion**: Es la parte más importante que no queda escrita en ningún lugar. Es la parte no visible. Es el momento en el cual uno dialoga con el PO, o quien conoce esta interfaz en el equipo de desarrollo, para captar las necesidades del problema
- **Confirmacion:** Condiciones que se tienen que dar para dar por satisfecha la US. Esta dentro de la Card.
- **Carta**:  Es la parte visible de la US. Es donde escribimos o expresamos algo que nos indique cual es la historia de usuario, de qué se trata y su valor del negocio

# Estructura de una US

COMO <NOMBRE DE ROL> YO PUEDO <ACTIVIDAD> DE FORMA TAL QUE <VALOR DE NEGOCIO QUE RECIBO>

El **nombre de rol** representa quién está realizando la acción o quién recibe el valor/beneficio de la actividad/acción; la **actividad** representa la acción que realizará el sistema y el **valor de negocio que recibo** comunica porque es necesaria la actividad.

# Modelado De Roles

Intentamos modelar los roles dentro del sistema para lograr identificar aquellos que probablemente tengamos en las US, y a partir de ellos encontrar en si las US

- TARJETA DE ROL DE USUARIO: Definimos un nombre especifico y su explicación en la cual
  detallamos su función.

También debemos evitar el uso de usuarios representantes como:
- Gerentes de Usuarios
- Gerentes de Desarrollo
- Alguien del grupo de marketing
- Vendedores
- Expertos del Dominio
- Clientes
- Capacitadores y personal de soporte

# Criterios de Aceptacion

Los criterios de aceptación son condiciones de satisfacción identificadas en el sistema y que le definen límites a la User Story. Nos permiten profundizar en las cuestiones que tienen que estar incluidas en la US para que la misma sea aceptada por el cliente y para que la US provea valor al negocio.

Estos son independientes de la implementación, es decir que están definidos desde la perspectiva del usuario y no están ligados a cómo se desarrolla internamente la funcionalidad.

Los criterios de aceptación ayudan a su vez a los desarrolladores a saber cuándo parar de agregar funcionalidad y a los testers a derivar las pruebas pertinentes

# Pruebas de Aceptacion

Las pruebas de aceptación se encuentran en el dorso de la tarjeta, complementan la historia, y también expresan detalles de la conversación. Nos dan la confirmación de que finalmente lo que nosotros estamos haciendo, si todas las pruebas que nosotros definimos las pasan, va a hacer lo que el PO espera. Agregamos también lo que esperamos que suceda cuando ejecutamos la historia de usuario. Entre paréntesis agregamos PASA o FALLA refiriéndonos al resultado del test

# Modelo INVEST

Es una manera de trabajar sobre la definición de Ready que propone que una serie de consideraciones a cumplir por la US:

- Independiente: se deben poder implementar en cualquier orden sin depender de que otra esté o no implementada.
- Negociable: define qué hacer pero no el cómo hacerlo.
- Valiosa: debe tener un valor concreto para el cliente.
- Estimable:Tengo que poder definir cuánto esfuerzo me va a llevar hacer la US.
- Pequeña (Small): para poder ser implementada en una iteración.
- Testeable: debe poder demostrar que fue implementada.

# Niveles de Abstraccion

Podemos tener otros tipos de US, como las Épicas y los Temas, que todavía no pueden ser incluidas en el Product Backlog, pero que podrían hacerlo en el futuro

- Las ÉPICAS son historias de usuario muy grandes, que en principio son así de grandes porque están en un lugar de la pila en donde todavía no fueron detalladas. Como todavía no debo implementarlas las dejo plasmadas como una gran US sabiendo que en algún momento voy a tener que llegar a disolverla, detallarla y hacerla más pequeña
- Las TEMAS podrían ser hasta más grandes que una épica. Es un conjunto de US relacionadas que defino en función de un título para recordarme que

todo lo que está incluido en ese tema lo voy a tener que tratar en algún momento. Tienen un nivel de abstracción hasta más grande que las épicas, ya que todavía no ha llegado el momento de detallarlas y no solo son ideas concretas si no que iniciativas para una propuesta de valor

- SPIKES: Un tipo especial de US son las spikes que sirven para quitar riesgo o incertidumbre de otro requerimiento, de otra US o de alguna faceta del proyecto que nosotros queremos investigar
- Las SPIKES TÉCNICAS son utilizadas para investigar enfoques técnicos en el dominio de la solución. Cualquier situación en la que el equipo necesite una comprensión más fiable sobre alguna tecnología a aplicar antes de comprometerse a desarrollar una nueva funcionalidad en un tiempo fijo
- Las SPIKES FUNCIONALES son utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema

# Estimaciones De Software

Estimar es predecir en un momento donde no tenemos gran conocimiento, donde hay incertidumbre

Por definición una estimación no es precisa, tiene un alto riesgo de incertidumbre y si estamos en el inicio del proyecto aún más todavía. Las estimaciones deben ir ajustándose a medida que avanzamos en el desarrollo del proyecto, para volverlas más precisas

Cuanto más al inicio del proyecto estamos, mayor es la incertidumbre y mientras vamos avanzando en el desarrollo del mismo, la incertidumbre va disminuyendo, si avanzamos en el proyecto y tenemos más información, tenemos que ajustar las estimaciones para que sean más precisas. No hay que esperar a tener toda la información que necesito para hacer la primera estimación.

Estimar NO es planear y no necesariamente nuestro plan tiene que ser lo mismo que lo estimado. Si bien la estimación sirve como base para planificar, los planes no tienen que seguirla, las estimaciones no son compromisos

En estimaciones ágiles puntualmente, se va a trabajar con las features o stories.

Las features/stories van a ser estimadas usando una medida de tamaño relativo

conocida como STORY POINTS (SP). Los story points son una ponderación que se le da a la historia de usuario cuyo peso es la combinación de su incertidumbre, esfuerzo y complejidad. Para esto se define una historia de usuario canónica que le servirá al equipo para estimar el resto en comparación a esta

- Lo primero que estimamos es el **TAMAÑO DEL SOFTWARE.**
  Con tamaño del software nos referimos a que tan grande va a ser el trabajo que vamos a hacer. Para esto utilizamos la técnica de CONTAR
  - Contar funcionalidades
  - Contar requerimientos
  - Contar la cantidad User Stories (estableciendo una métrica de conteo)
  - Contar la cantidad de Casos de Uso
- Otra de las estimaciones que realizamos es el **ESFUERZO**.
  - Con esfuerzo nos referimos a la cantidad de horas lineales que voy a necesitar para construir el software, solo CUENTO, cuantas horas lineales de desarrollo voy a necesitar.
- Luego de estimar el esfuerzo, estimo el **CALENDARIO**
  - Es justamente calendarizar el esfuerzo y proponer una fecha de entrega.

Una vez que logramos estimar tamaño, esfuerzo y tiempo calendario, vamos a empezar a trabajar con **COSTOS** y **PRESUPUESTOS**

No debemos confundirnos, tamaño NO es esfuerzo, ya que esfuerzo tiene que ver con horas lineales

# Tecnicas de Estimacion

- **METODOS BASADOS EN LA EXPERIENCIA**: Esta técnica se basa en la experiencia pasada del
  equipo en proyectos similares para determinar la estimación de esfuerzo y tiempo requerido para un nuevo proyecto.
- **METODOS BASADOS EXCLUSIVAMENTE EN RECURSOS:** Esta técnica se basa en la cantidad y el tipo de recursos (personas, tiempo, herramientas, etc.) necesarios para llevar a cabo un proyecto.

- **METODOS BASADOS EXCLUSIVAMENTE EN EL MERCADO:** Esta técnica se basa en el
  costo de proyectos similares en el mercado para determinar la estimación de esfuerzo y tiempo requerido para un nuevo proyecto.
- **METODOS BASADOS EN LOS COMPONENTES DEL PRODUCTO O EN EL PROCESO DE DESARROLLO:** Esta técnica se basa en la identificación y descomposición de los componentes
  del software y del proceso de desarrollo para estimar el esfuerzo y tiempo requerido para cada componente.
- **METODOS ALGORITMICOS**: Esta técnica se basa en modelos matemáticos y estadísticos para
  determinar la estimación de esfuerzo y tiempo requerido para un proyecto.

# Velocity

Es una medida (métrica) del progreso de un equipo. Se calcula a partir de la suma de los Story Points asignados a cada User Story que el equipo completó al 100% durante la iteración que se está midiendo.

Esta métrica o medida nos sirve para corregir errores de estimación gracias a la gran cantidad de información que nos brinda.

# Metodos de Estimacion

**Poker Estimation**:  Técnica de estimación basada en la colaboración y la experiencia del equipo.
Todo el equipo de desarrollo se reúne, se define la US canónica y luego para cada US a estimar se
vota de manera anónima cuantos story points le asignaría cada integrante.

Si todos los estimadores seleccionan el mismo valor, ese es el estimado. Sino, los estimadores discuten sus resultados, poniendo especial atención en los más altos y los más bajos. Los valores de las cartas son los de la serie de Fibonacci ya que se considera que la complejidad de las historias de usuario tiene un crecimiento exponencial.

# Gestion de Productos

Cuando hablamos de gestión de producto empezamos a pensar en términos de visión de la organización, para qué construimos productos de software. En este ámbito tenemos un objetivo, ya sea satisfacer a los clientes, tener muchos usuarios, obtener ganancias, etc.

Es común en los productos de software construir características que no se usan o se usan muy raramente, lo que resulta en un esfuerzo desperdiciado. Es por ello que debemos estudiar qué es lo que realmente es de valor para el cliente con el objetivo de que el desperdicio tienda a cero

En principio nos vamos a imaginar una hipótesis o un valor que tiene el producto o servicio que queremos desarrollar.  Si nosotros a raíz de esta hipótesis desarrollamos un producto completo con muchas funcionalidades y recursos utilizados y de ahí lo comercializamos, corremos el riesgo de que el producto no sea algo que el mercado o los usuarios estén demandando

Es por esto que buscamos una forma de minimizar el esfuerzo para poder construir un producto que nuestro usuario quiere

# MVP (Producto Minimo Viable)

La idea del MVP es trabajar con lo mínimo del producto que necesito obtener. Lo mínimo que tendría que tener el producto para que los (posibles) usuarios del mismo lo puedan evaluar y me puedan decir si el producto les atrae o no

El objetivo del MVP es comprobar que la hipótesis del producto a construir funciona o si tengo que hacer cambios. La idea es iniciar un circuito de aprendizaje a partir de la participación de los usuarios y la retroalimentación sobre si el producto cubre expectativas

- Puede suceder, como bien dijimos, que así como está definida la hipótesis sirva, y que todas las personas que vean ese MVP se sientan a gusto y expresen que lo necesitan.
- También puede ocurrir que cada potencial cliente o usuario, al verlo, pida funcionalidades diferentes. Si con cada potencial cliente o usuario que prueba el MVP recibo un feedback distinto, quiere decir que no está tan claro el mercado en el cual este producto va a funcionar. Esto implica que tengo que seguir trabajando sobre la visión del producto para encontrarla.
- Otra alternativa que podría suceder es que, en algún punto, la hipótesis planteada tenga algunas variaciones. En este contexto, la hipótesis puede moverse o modificarse según la exigencia del mercado.

# MMF (Carcateristicas Minimas Comerciables)

Se enfoca en definir los elementos mínimos que deben estar presentes en un producto o servicio para que sea comercializable. Es la validación del producto en términos de la comercialización

La comercialización Ya no estoy parado en una hipótesis para validar si el producto tiene mercado o no, sino que estoy definiendo cual es la pieza mínima que voy a tener que construir y comercializar para que el producto sea rentable. Esto es ya que, de nuevo, no tiene sentido construir el producto completo hasta no ver si va a funcionar su comercialización

También es importante construirlo con el mínimo necesario para sacarlo rápidamente a la venta cuando el producto es atractivo y todavía no está en el mercado, ya que, cuanto más tiempo pase, mayor es el riesgo de que la idea sea implementada antes.

# MVF (Caracteristica Minima Variable)

Se trata de la característica mínima que se puede construir e implementar rápidamente, utilizando recursos mínimos, para probar la utilidad de la misma

Quiero validar cual es la característica que es la que hace la diferencia y que va a hacer en primera medida que compren mi producto, siendo una feature que agrega valor al producto

Si la MVF resulta exitosa, se pueden desarrollar más MMF en esta área para tomar ventaja

**Dilema de la Audacia de cero**

Refiere a la estrategia de empezar desde cero en lugar de intentar construir sobre una base pequeña. Debemos pensar en término de la construcción de un producto en grande, aunque resulte más riesgoso. Si se pospone la experimentación con el MVP, van a surgir resultados como una gran cantidad de trabajo desperdiciado

Es por esto que debemos utilizar el MVP para experimentar con los primeros usuarios en el mercado probando todos los elementos disponibles comenzando por los más riesgosos

**SUPUESTOS DE “SALTOS DE FE”**

Los elementos más riesgosos del plan / concepto de un nuevo producto o stratup se denominan supuestos de salto de fe. Dentro de estos elementos encontramos supuestos sobre si el producto realmente entrega valor una vez que se está utilizando, o cómo creemos que los clientes descubrirán nuestro producto.

**LA FASE CONSTRUIR DEL MVP**

Es importarte saber que cuando construimos el MVP, su complejidad puede variar en complejidad, desde anuncios, explicaciones simples, hasta prototipos tempranos. La idea es tener rápidamente y con el menor esfuerzo tener el MVP para poder validarlo y retroalimentar la hipótesis para entrar en el ciclo de aprendizaj

Siempre que nos entremos en dudas podemos:

- En caso de duda, simplificar
- Evitar la construcción excesiva y la promesa excesiva
- Saber que cualquier trabajo adicional más allá de lo que necesita podría ser un desperdicio

# Construcciones de un MVP

Es importarte saber que cuando construimos el MVP, su complejidad puede variar en complejidad, desde anuncios, explicaciones simples, hasta prototipos tempranos. La idea es tener rápidamente y con el menor esfuerzo tener el MVP para poder validarlo y retroalimentar la hipótesis para entrar en el ciclo de aprendizaje. Cuando hablamos de MVP empiezan a surgir un montón de problemáticas sobre cómo hacemos para minimizar el desperdicio y el esfuerzo al construir el MVP. Para esto debemos:
- En caso de duda, simplificar
- Evitar la construcción excesiva y la promesa excesiva
- Saber que cualquier trabajo adicional más allá de lo que necesita para comenzar el ciclo podría ser un desperdicio

# SCRUM

Scrum es un marco ligero (simple y flexible) que ayuda a las personas, equipos y organizaciones a

generar valor a través de soluciones adaptables para problemas complejos. Scrum requiere un Scrum Master para fomentar un entorno donde:

1. Un Product Owner ordena el trabajo de un problema complejo en un Product Backlog.
2. El equipo de Scrum convierte una selección del trabajo en un incremento de valor durante un
   Sprint.
3. El equipo de Scrum y sus partes interesadas (stakeholders) inspeccionan los resultados y
   realizan los ajustes necesarios para el próximo Sprint.
4. Repetir

Es un framework empírico alineado a los principios y valores ágiles.

Scrum se basa en el empirismo y el pensamiento Lean. El empirismo afirma que el conocimiento
proviene de la experiencia y la toma de decisiones basadas en lo que se observa. El pensamiento Lean reduce los desperdicios y se centra en lo esencial. Scrum emplea un enfoque iterativo e incremental para optimizar la previsibilidad y controlar el riesgo

# Pilares del empirismo de Scrum

● **Transparencia**: el proceso y el trabajo deben ser visibles para aquellos que realizan el trabajo, así como para los que reciben el trabajo.
● **Inspección**: los artefactos y el progreso deben ser inspeccionados con frecuencia para detectar varianzas o problemas.
● **Adaptación**: el proceso que se está aplicando o el producto resultante deben ajustarse ante
desviaciones fuera de los límites aceptables

# Valores de Scrum

- **Compromiso**: a lograr los objetivos y apoyarse mutuamente.
- **Enfoque**: hacia el trabajo del Sprint para hacer el mejor progreso posible en dirección a los

objetivos.

- **Apertura**: ser transparente y honesto en todo lo relacionado con el trabajo del equipo.
- **Respeto**: entre los miembros del equipo por ser personas capaces e independientes.
- **Coraje**: valor para hacer lo correcto y para trabajar con problemas complejos.

# Equipo Scrum

Scrum involucra a grupos de personas que conjuntamente tienen todas las habilidades y experiencia para hacer el trabajo y compartir o adquirir tales habilidades según sea necesario.
La unidad fundamental de Scrum es un pequeño equipo de personas. Es una unidad fusionada de profesionales enfocadas en un objetivo para el producto. El equipo de Scrum es lo suficientemente pequeño como para permanecer ágil y lo suficientemente grande como para completar un trabajo significativo dentro de un Sprint, por lo general 10 o menos personas.

- **Scrum Master**: Responsable de establecer Scrum. Lo consigue ayudando a todos a comprender la teoría y la práctica de Scrum, tanto dentro del equipo como en toda la organización
- **Product Owner:** Es responsable de maximizar el valor del producto resultante del trabajo del equipo de Scrum. Es quien conoce los lineamientos del negocio, por lo que es quien escribe las historias de usuario y dice qué valor y prioridad tiene cada una
- **Desarrolladores:** Se comprometen a crear cualquier aspecto de un incremento útil y funcional
  en cada sprint. Este término incluye a todas las personas que contribuyen directamente a la
  creación del producto

# Eventos de Scrum

# Sprint

El Sprint en Scrum es un período de tiempo fijo, usualmente de un mes o menos, donde se lleva a cabo parte del trabajo necesario para alcanzar un objetivo del producto. Durante el Sprint, no se hacen cambios que pongan en peligro el objetivo del sprint, la calidad no disminuye y se puede ajustar el alcance según sea necesario

# Sprint Planning

El Sprint Planning inicia el Sprint estableciendo el trabajo que se realizará para el mismo.

Este plan se hace de manera conjunta por todo el equipo Scrum y el Product Owner el cual aborda los siguientes temas:

- Qué valor aporta este Sprint
- Qué se puede hacer en este Sprint
- Cómo se realizará el trabajo

# Daily Scrum

Evento de 15 minutos que tiene como objetivo inspeccionar el progreso hacia el objetivo del sprint y adaptar el sprint backlog según sea necesario, produciendo un plan accionable para el día siguiente de trabajo.

# Sprint Review

Tiene como objetivo inspeccionar el resultado del sprint y determinar futuras adaptaciones.

El equipo presenta los resultados de su trabajo y se discute, junto a los stakeholders, el progreso hacia el objetivo del producto. En base a esta información y a nuevas necesidades u oportunidades, se evalúa qué hacer a continuació

# Sprint Retrospective

El equipo Scrum se reúne para reflexionar sobre el Sprint que acaba de concluir y discutir qué salió bien, qué puede mejorarse y cómo pueden mejorar sus procesos y prácticas para futuros Sprints

# Artefactos

# Product Backlog

Lista en forma de pila de lo que se necesita para mejorar el producto. Funciona como fuente para el trabajo del equipo. En esta lista se descomponen los elementos de trabajo pendiente del producto en tareas más pequeñas y detalladas

# Sprint Backlog

Conjunto de elementos seleccionados del Product Backlog para el Sprint, más un plan accionable para entregar el incremento. Es un plan creado por y para los desarrolladores, en el que se describe qué se hará el el sprint para lograr el objetivo del mismo

# Incremento

Paso concreto hacia el Objetivo del Producto, aditivo a incrementos anteriores y verificado a fondo asegurando que todos los incrementos funcionen juntos. Para proporcionar el valor, el incremento debe ser utilizable

# Capacidad del Scrum

La capacidad de un equipo en un Sprint se refiere a la cantidad total de trabajo que el equipo puede completar durante un Sprint.

Esta capacidad depende de varios factores, incluyendo la disponibilidad del equipo, la complejidad de los trabajos pendientes, y cualquier compromiso externo.
Es una estimación que en Scrum se considera una métrica, es una medida dinámica que necesita ser ajustada y considerada con atención para garantizar la efectividad y eficiencia del equipo

# Ambiente de traba Scrum

El ambiente de trabajo en Scrum debe ser abierto y colaborativo, donde la comunicación fluye

libremente, evitando el silencio absoluto que puede indicar problemas no resueltos.

Pizarrones y otras herramientas visuales son esenciales para proporcionar transparencia y facilitar la planificación y el seguimiento del progreso. Además, la flexibilidad en la definición de horarios por parte del equipo promueve la autonomía, el compromiso

# Cadencia Scrum

En Scrum, la cadencia se refiere a la regularidad y el ritmo constante de los eventos y entregas,
estableciendo un flujo predecible y sostenible de trabajo. Los Sprints tienen una duración fija y
repetitiva, con eventos estructurados como la planificación, las reuniones diarias, la revisión y la
retrospectiva, que ocurren a intervalos regulares (time box). Esta cadencia asegura la entrega
continua de valor.

# Gestion De Configuracion De Software (SCM)

SCM es una actividad de soporte transversal a todo el proyecto que aplica dirección y monitoreo
administrativo y técnico, y cuyo propósito es mantener la integridad del producto a lo largo de todo el ciclo de vida

Los sistemas de software siempre cambian durante su desarrollo y uso. Conforme se hacen cambios al software, se crean nuevas versiones del sistema. Los sistemas pueden considerarse como un conjunto de versiones donde cada una de ellas debe mantenerse y gestionarse. Estos cambios en el software tienen su origen en cambios del negocio y nuevos requerimientos

Ante estos cambios, es ideal que haya integridad en el producto de software y a través de SCM logramos esto.

El propósito de la SCM es también establecer la integridad de los productos y esto involucra:

1. Identificar características técnicas y funcionales de ítems de configuración

2. Documentar características técnicas y funcionales de ítems de configuración
3. Controlar los cambios de esas características identificadas y documentadas
4. Registrar y reportar estos cambios
5. Verificar correspondencia con los requerimientos (trazabilidad).

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen

Decimos que se mantiene la integridad de un producto de software cuando:

1. Satisface la ncecesidades del usuario
2. Permite la rastreabilidad durante todo el ciclo de vida
3. Satisface criterios de Performance y RNF
4. Cumple con expectativas de costos

# Items de Configuracion

Son aquellos artefactos que forman parte del producto o proyecto, y pueden ser almacenados en un repositorio, sin importar su extensión/tipo.

Pueden sufrir cambios, y se desea conocer su estado y evolución a lo largo del ciclo de vida. Es decir, es cualquier aspecto asociado al producto de software (requerimientos, diseño, código, datos de prueba, documentos, etc.) que es necesario mantener.
Son todos aquellos elementos que componen toda la información producida como parte del proceso de ingeniería de software, como ser programas de computadora (código fuente y ejecutables), documentos que describen los programas (documentos técnicos o de usuario), datos (de programa o externos), etc.

Los ítems, dependiendo de su naturaleza, pueden durar lo que dure el ciclo de vida del proyecto, producto o sprint.

# Repositorio

Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos. Puede ser una o varias bases de datos.

Tiene una estructura para mantener el orden y la integridad y el que tenga una estructura ayuda a la seguridad, los controles de acceso, las políticas de Backup y todo aquello que se aplica sobre un repositorio.

- CENTRALIZADO: está caracterizado porque un servidor contiene todos los archivos con sus
  versiones. Los administradores tienen un mayor control sobre el repositorio. Tiene como desventaja qué si falla el servidor, todo lo positivo se cae
- DESCENTRALIZADO: está caracterizado porque cada cliente tiene una copia exactamente igual del repositorio completo. Tiene como ventaja que se resuelve el problema de los servidores centralizados, debido a que si el servidor falla sólo es cuestión de “copiar y pegar”, además posibilita otros workflows no disponibles en el modelo centralizado

# Version

Instancia de un ítem de configuración que difiere de otras instancias de este ítem las cuales se identifican unívocamente y controlar una versión refiere a la evolución de un único ítem de configuración.

Cada versión de software es una colección de elementos de configuración. La versión es un punto particular en el tiempo de ese ítem de configuración, es decir, su estado.

Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado.

# Variante

Una variante es una versión de un ítem de configuración (o de la configuración) que evoluciona por separado. Las variantes representan configuraciones alternativas las cuales varian segun el ambiente en donde estas se instalan

# Configuracion de Software

Conjunto de todos los ítems de configuración con su versión específica. Define una foto de los ítems de configuración en un momento de tiempo determinado.

Una configuración es el conjunto de todos los componentes fuentes que son compilados, sus documentos y la información de la estructura que definen una versión del producto a entregar. Siendo esta la sumatoria de todos los ítems de configuración que tiene en un momento determinado

# Linea Base

Es un conjunto de ítems de configuración que han sido construidos y revisados formalmente, de manera que pueden ser tomados como referencia para demostrar que se ha alcanzado cierto nivel de madurez en ellos, y que sirve como base para desarrollos posteriores.

Esto es lo mismo que decir que es una configuración de software que ha sido formalmente revisada y aprobada, que sirve como base para desarrollos futuros.

Este conjunto de ítems debe tener una referencia única, y esto se hace a través de “tags”.
Se acuerdan parámetros para determinar qué se considera o qué debe cumplir los ítems de configuración para considerarse como línea base.

Si se desea cambiar una línea base, existe un protocolo formal de control de cambios, dirigido por el Comité de Control de Cambios, que permite definir el procedimiento a seguir para manejar peticiones de cambios

- De especificación: (Requerimientos, Diseño) son las primeras línea base, dado que no cuentan con código. Podría contener el documento de especificación de requerimiento.
- Operacionales: contiene una versión de producto cuyo código es ejecutable, han pasado por un control de calidad definido previamente.

# Ramas

Es un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, por varios motivos, ya sea experimentación, resolución de errores en el desarrollo o para desarrollar un mismo software para distintas plataformas.

# Actividades de SCM

# Identificacion de Items de Configuracion

Esto implica una identificación unívoca para cada ítem, donde en el equipo se definirán políticas y reglas de nombrado y versionado para todos ellos.

También, se debe definir la estructura del repositorio, y la ubicación de los ítems de configuración dentro de esa estructura, lo que permite a los desarrolladores controlar y velar por la integridad del producto, como así también a los clientes evaluar esa integridad

- **Ítems de producto:** tienen el ciclo de vida más largo, y se mantienen mientras el producto exista
- **Ítems de proyecto:** el plan de proyecto, el listado de defectos encontrados, tienen un ciclo de vida de proyecto
- **Ítems de iteración:** Conocer el ciclo de vida de un ítem permite establecer la nomenclatura de este

# Control de cambios

Tiene su origen en un requerimiento de cambio a uno o varios ítems de configuración que se encuentran en una línea base.

Es un Procedimiento formal que involucra diferentes actores y una evaluación del impacto del cambio.

Una vez definida la línea base, no es posible cambiarla sin antes pasar por un proceso formal de control decambios. Es decir que el control se hace sobre los ítems de configuración que pertenecen a la línea base, ya que todos los trabajadores del software tienen a dichos ítems como referencia

# Comite de Control de Cambios

Este proceso es llevado a cabo por el comité de control de cambios, el cual se reúne para autorizar la creación y cambios sobre la línea base

1. Se recibe una propuesta de cambio sobre una línea base determinada

2. El comité de control de cambios realiza un análisis de impacto del cambio para evaluar el esfuerzo técnico, el impacto en la gestión de los recursos, los efectos secundarios
3. En caso de que se autorice la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar
4. Luego de realizado el cambio, el comité vuelve a intervenir para aprobar la modificación de la línea base y marcarla como línea base nuevamente
5. Finalmente se notifica a los interesados los cambios realizados sobre la línea base.

# Auditorias De Configuracion

Son controles autorizados a realizar por el equipo de desarrollo en un momento determinado, donde un auditor externo al equipo (independiente y objetivo) analiza las líneas base, las cuales permiten “congelar” y analizar en un momento determinado cuál es el estado del software, y si se están cumpliendo todas las pautas que plantea esta disciplina de SCM

La auditoría de configuración se hace mientras el producto se está construyendo y su objetivo es que el producto tenga calidad e integridad. Se entiende que lo más barato es prevenir

- **Auditoría física de configuración (PCA):** Asegura que lo que está indicado para cada ICS en la línea base o en la actualización se ha alcanzado realmente.
- **Auditoría funcional de configuración (FCA):** Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos

# Informes de Estado

Provee un mecanismo para mantener un registro de cómo evoluciona el sistema, y dónde está ubicado el software, comparado con lo que está publicado en la línea base.

Esto permite mantener a todo el equipo informado sobre la última línea base, y que se trabaje en base a su última versión, y no sobre una versión obsoleta.

Estos reportes incluyen todos los cambios que han sido realizados a las líneas base durante el ciclo de vida del software. Normalmente esto consta de grandes unidades de datos, por lo que se utilizan herramientas automatizadas por una computadora.

Permite responder a preguntas como:

- ¿Cuál es el estado del ítem?
- ¿La propuesta de cambio fue aceptada o rechazada por el comité?
- ¿Qué versión de ítem implementa un cambio de una propuesta de cambio aceptada?
- ¿Cuál es la diferencia entre dos versiones de un mismo ítem?

# Plan De Gestion De Configuracion

El Plan de Gestión de Configuración es un documento que establece cómo se gestionará y controlará la configuración del software a lo largo de su ciclo de vida. En él se deben incluir reglas y convenciones, procedimientos formales, etc. Este plan garantiza que la configuración del software se mantenga controlada, trazable y consistente a lo largo del ciclo de vida del proyecto

# SCM en Ambientes Agiles

La gestión de configuración en ambientes ágiles se adapta a los principios y prácticas ágiles para
servir al equipo de desarrollo y facilitar la entrega continua de valor al cliente. El objetivo es
responder de manera rápida y eficiente a los cambios en los requisitos del proyecto, adaptando la
configuración del software de manera oportuna.

- Seguimiento y coordinación del desarrollo en lugar de controlar a los desarrolladores
- Automatización siempre que sea posible: las herramientas de automatización pueden gestionar automáticamente las versiones del software
- Eliminar el desperdicio: reducir la cantidad de trabajo innecesario y los procesos que no

añaden valor al proyecto

- Feedback continuo: recibir comentarios de los desarrolladores sobre las prácticas de gestión
de la configuración

# Practicas Continuas

- **Integracion Continua**: Es una práctica de desarrollo que promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día. Cada integración de código es luego verificada por una serie de pruebas automatizadas, permitiéndole al equipo detectar problemas de manera temprana. La integración continua es asegurar que el software pueda ser desplegado en cualquier momento
- **Entrega Continua:** Es una disciplina de desarrollo de software en la que el software se construye de tal manera que puede ser liberado en producción en cualquier momento de modo que el software en desarrollo siempre esté en un estado que se pueda desplegar en producción manualmente
- **Despliegue Continuo:** El objetivo principal del despliegue continuo es entregar nuevas funcionalidades, mejoras y correcciones de manera rápida, frecuente y confiable a los usuarios finales. Esto permite a las empresas responder rápidamente a las necesidades del mercado y mejorar continuamente su software.